

Grading and Feedback

Course: REI603M The AI Lifecycle

Assignment: Assignment 4 - Prototype Sprint

Students: jre5, sos106, sbs87

Project: Ratatoskur (AI Math Coach)

Grade: 10/10

The team delivered an ambitious native iOS + FastAPI prototype — SwiftUI with PencilKit handwriting input, FastAPI backend with PostgreSQL (Neon) and JWT authentication, Gemini 3 Flash Preview with vision API for handwriting analysis, Langfuse observability, Cloudflare R2 storage, and 20 test cases evaluated through the full prototype flow (95% verdict accuracy). The technical scope is the broadest of all submissions.

Checklist & Grade Computation

Part 1: Working Prototype

- [x] Interactive interface – Native iOS SwiftUI application (“Ratatoskur”) with PencilKit canvas for handwriting input, problem creation flow, three interaction modes (Hint, Check Solution, Reveal), attempt history, and tutor response display. Full authentication flow (register/login/refresh/logout).
- [x] Real LLM integration – Google Gemini 3 Flash Preview via `google-genai` SDK (v1.62.0). Vision API for multimodal input: receives both a problem image and a handwritten solution image. Structured JSON output with verdict, response type, and Icelandic-language feedback.
- [x] Refined prompts from A3 – Single production prompt (`backend/prompt.txt`, 111 lines) incorporating 3-phase analysis (visual analysis → mathematical analysis → response generation), 5 examples, error classification taxonomy (conceptual, calculation, logic, incomplete), and mode-specific response rules (`check_solution`, `hint`, `reveal`). Evolved from A3 prompt experiments.
- [x] Prompt management – Prompt stored as a separate file (`backend/prompt.txt`), not inline. Loaded at runtime. File-based management allows versioning via git.
- [x] Basic error handling – API: FastAPI exception handlers with structured error responses, JWT token validation with proper error codes, file upload validation. Backend: try-catch on Gemini API calls, timeout handling. Frontend: error state display in SwiftUI, network error handling in API client. No raw stack traces exposed to users.
- [x] Environment management – `.env.example` in `backend/` with `DATABASE_URL`, `JWT_SECRET`, `GEMINI_API_KEY`, R2 credentials, and optional Langfuse keys. `.gitignore` excludes `.env`. JWT configuration in `config.py` with environment variable overrides.
- [x] Git repository with README – Comprehensive README (4.6 KB) with architecture overview, both backend and frontend setup instructions, API endpoint documentation, cost methodology, and evaluation artifact locations.

Part 2: Observability & LLMOps

- [x] Every LLM call logged – Langfuse integration (v3.14.1) with trace/observation export. 76 observations across 37 generation calls tracked in `langfuse_observations_export.csv`.
- [x] Latency tracking – Average latency: 24.8 seconds. P95 latency: 87.7 seconds. Measured from span-level latency data across 37 generation calls. Latency reported in both span and generation metadata.
- [x] Token usage and cost tracking – Total: 136,816 input tokens + 4,710 output tokens + 77,136 thought tokens = 218,662 total tokens. Estimated cost: \$0.31 total, \$0.0085 per interaction. Cost

methodology documented in README using Gemini 3 Flash pricing (\$0.50/1M input, \$3.00/1M output including thought tokens).

- [x] Observability tool integrated – Langfuse with exported observations CSV. Metrics computation script (`compute_langfuse_metrics.py`) generates summary statistics. Separate summary markdown file.
- [x] Trace shown during demo – Demonstrated Langfuse trace walkthrough and metrics during presentation.

Part 3: Evaluation Integration

- [x] At least 20 test cases run against prototype – 20 test cases from the A3 evaluation dataset run through the full prototype flow. Each test case includes a problem image and handwritten solution image processed through the complete SwiftUI → FastAPI → Gemini pipeline.
- [x] Comparison to A3 baseline – A4 verdict accuracy: 95.0%. A3 v4 subset accuracy (same 20 images): 95.0%. Delta: 0.0 — no degradation through the application layer. Non-feasible ratio: 0.0% for both.
- [x] Failure analysis – 1 mismatch identified: test case 48 (Ambiguities category, expected verdict “unclear”, received “correct_so_far” with response_type “explanation”). The model failed to flag an ambiguous input as unclear, instead proceeding with a mathematical analysis. This is a model-level failure: the visual analysis phase did not detect the ambiguity. Detailed mismatch breakdown in `assignment4_mismatches.csv`.

Part 4: Reflection

- [x] Architecture decisions discussed – SwiftUI + PencilKit choice, FastAPI backend, Neon Postgres, Cloudflare R2, JWT auth design.
- [x] PRD vs. reality discussed – Covered in presentation.
- [x] Team work split discussed – Covered in presentation.
- [x] LLM-assisted development discussed – Covered in presentation.

Presentation

- [x] Slides covering prescribed structure – PDF slides submitted (`REI603M_Assignment4_presentation.pdf`).
- [x] Live demo shown – Demonstrated iOS app with handwriting input and tutor response.
- [x] Slides submitted in advance – PDF included in submission.
- [x] Source code included – Both `ai_lifecycle/` (backend + artifacts) and `ai_lifecycle_frontend/` (SwiftUI) directories with full source. 23 Python backend files, 31 Swift frontend files.

Deductions: None

Final grade: 10 - 0 = 10/10

—

Strengths

The multimodal pipeline — PencilKit handwriting → Cloudflare R2 upload → Gemini vision analysis of both problem and solution images — solves the real UX problem of math tutoring: students work with handwritten equations that need to be understood in context, not just typed text. The 3-phase prompt (visual analysis → mathematical analysis → response generation) prioritizes readability before math evaluation — unclear handwriting triggers clarification rather than guessing, which is pedagogically correct.

The mode-based interaction (hint, check_solution, reveal) maps to how students learn: check if on track, get a nudge, or see the full answer. Each mode has strictly defined prompt behavior — hints are Socratic questions, check_solution identifies the first error without fixing it, reveal gives step-

by-step solutions. The error taxonomy (conceptual, calculation, logic, incomplete) helps students understand **how** they're wrong.

JWT auth with refresh tokens, Cloudflare R2, and Neon PostgreSQL show production infrastructure thinking. Icelandic-language output with mathematical terminology guidance is well-integrated into the prompt.

Areas for Improvement

P95 latency of 87.7 seconds is too high for interactive tutoring. Waiting over a minute for feedback breaks the learning flow. The bottleneck is likely the vision API processing two images with a thinking model. Stream the response so partial output appears while processing continues, or use a fast model for image-to-text conversion before running the full analysis on text.

Single prompt file limits experimentation. One `backend/prompt.txt` file means no versioning, no A/B testing, and no record of which version produced which results. At minimum, use numbered files (`prompt_v1.txt`, `v2.txt`, ...). Better: use Langfuse prompt management since you already have it integrated.

Ambiguity detection gap (test case 48). Phase 1 catches unclear handwriting but not semantic ambiguity (readable notation with multiple mathematical interpretations). Add examples to the prompt: "If notation could be interpreted two ways (e.g., $f'(x)$ vs. $f(x)'$, or dx/dy vs. dy/dx), set verdict to unclear and present both interpretations."

Teacher's Comments & Discussion Notes

"Great to use SwiftUI, it's not cross platform, but it's a great framework for building UIs. I appreciate using the PencilKit."

PencilKit gives you pressure-sensitive, low-latency ink rendering — important for math notation where strokes, superscripts, and subscripts need precise capture. A web canvas would produce a significantly worse experience. The trade-off is iOS-only reach. For the final project, decide whether to add a web fallback (typed-LaTeX input for non-iPad users) or double down on native. Either way, make it a conscious product decision in the PRD.

"Think of how you can organize the prompts in a manner that makes it easier to change."

Three approaches in order of sophistication: (1) **Versioned files:** `prompts/v1.txt`, `v2.txt`, etc. with a `PROMPT_VERSION` env var. Simple, git-tracked, easy to diff. (2) **Modular prompts:** Split into sections (`visual_analysis.txt`, `math_analysis.txt`, `response_rules.txt`, `examples/`) assembled at runtime. (3) **Langfuse prompt management:** You already have Langfuse integrated — use its prompt versioning for history, A/B testing, and deployment control without code changes. See [Langfuse Prompt Management](#).

"The p95 latency is really high. You may want to use streaming output if possible."

Three strategies:

(1) **Streaming:** SSE from FastAPI so partial output appears as it generates. The verdict can show in under 5 seconds, detail follows. Doesn't reduce total latency but transforms perceived latency.

(2) **Two-stage pipeline:** Fast vision model for image-to-text (2–5 seconds), then text-only model for reasoning (cheaper and faster than vision). Cache the problem image OCR so repeat queries only process the solution.

(3) **Progress indicators:** Show “Reading your handwriting...”, “Analyzing your solution...”, “Preparing feedback...” while the model processes 77k+ thought tokens.

“Think about the best UX flow for your application.”

Streamline the current flow:

- **Persistent canvas:** Let students draw, submit, revise, and resubmit in the same view rather than creating a new problem each time. Show previous attempts alongside the current canvas.
- **Quick retry:** After error feedback, one-tap to clear the problematic step and resubmit without navigating away.

“Consider whether you want to have the ability to interact further with a response.”

Currently each query is stateless. Multi-turn context would let students ask follow-ups like “Can you explain the chain rule again?” or “What should the chain rule give for this function?” Maintain a conversation array per problem, include previous responses in context for follow-ups. Voice input is also worth considering — a student with a stylus in hand may prefer speaking follow-ups. See [Apple Speech Framework](#).

“Think of whether you can improve the design of the UI.”

The app is functional but utilitarian. For secondary school students, visual polish affects engagement. Consider: approachable color schemes, animated transitions for feedback arrival, visual error highlighting on the canvas (red circle on the error step), and a progress dashboard for problems attempted/solved.

“If you have a fast model to convert images to text, then you might be able to respond more quickly.”

The most actionable latency fix. Instead of sending both images to Gemini for combined vision + reasoning, split into two stages: (1) fast vision model converts images to structured text (LaTeX or plain text), 2–5 seconds; (2) text-only model runs the full 3-phase analysis, which is faster than vision. Cache the problem image OCR so repeat queries only process the solution image.

—

Ideas for Further Improvement

1. **Two-stage pipeline (OCR + reasoning):** Fast vision model for handwriting-to-text, then text model for analysis. Cache problem OCR for repeat queries. Should cut p95 latency by 50%+.
2. **Streaming via SSE:** Stream partial output from FastAPI to iOS. Show verdict first, detail follows. See [FastAPI StreamingResponse](#).
3. **Multi-turn conversation per problem:** Maintain conversation history for follow-up questions about feedback.
4. **Canvas error highlighting:** Overlay visual indicators on the PencilKit canvas at the approximate error location. Map “step 3 has an error” to the corresponding ink strokes.
5. **Voice input for follow-ups:** Apple Speech Framework for verbal questions while hands are on the stylus. See [Apple Speech Recognition](#).
6. **Prompt versioning:** Replace single `prompt.txt` with versioned files or Langfuse prompt management. See [Langfuse Prompt Management](#).
7. **Adaptive difficulty:** Track performance across problems, estimate skill level, suggest appropriate difficulty. See [Item Response Theory](#).

8. **Problem library with curriculum alignment:** Curated problems mapped to the Icelandic secondary school curriculum, browsable by topic and difficulty.
9. **Offline mode:** Cache problem-solution-feedback triples locally. Sync to server when connectivity returns.
10. **Progress dashboard and gamification:** Problems attempted/solved, streaks, topic mastery chart. See [Swift Charts](#).
11. **LaTeX rendering in feedback:** Render math notation properly using [SwiftMath](#) or WKWebView with KaTeX/MathJax — currently LaTeX may display as raw text.
12. **Batch evaluation automation:** Run the 20-test suite on every prompt change. The existing `compute_assignment4_evaluation.py` is a good foundation.